Final Project (Group): Text-Based Adventure Game (Python)

Teams of 6, GitHub Collaboration, Required Cyber Pack

Overview

You will work in a team to design and build a text-based adventure game in Python. This final project is designed to give you real teamwork experience by collaborating through GitHub using issues, pull requests, code reviews, and shared ownership of one codebase.

Team Size

- **Teams of 6 students** (about 10 teams total)

Project Goal

Create **one complete text-based adventure game per team** that meets the required game features and includes a required cybersecurity element called the **Cyber Pack**.

Part 1: Required Game Features (Core Requirements)

1) Story and Branching

Your game must include:

- **At least 3 different story paths** (choices that lead to different outcomes)
- **At least 3 different endings** based on player choices
- A clear **beginning, middle, and end**
- **Example (3 Story Paths)**
    - Your game can start with the same opening scene, then split into different paths based on the player's choices.
    - Beginning (same for everyone):
        - You wake up inside a locked research facility. The emergency lights are on, and an alarm is looping. You need to escape.
    - Path 1: "Find the Keycard"
        - Choice: You decide to search offices for a keycard instead of going to the

server room.

        - Outcome: You meet an NPC guard, collect a keycard, unlock the exit door, and escape through the loading bay.

    - Path 2: “Hack the Terminal”

        - Choice: You go straight to a terminal to try to unlock doors.

        - Outcome: You solve a puzzle (password/login), access the map, disable one alarm zone, and escape through a controlled security checkpoint.

    - Path 3: “Help the Scientist”

        - Choice: You follow a voice call for help and choose to rescue a trapped scientist.

        - Outcome: The scientist gives you a hidden code and a special item, but you trigger a different enemy encounter. You escape through a maintenance tunnel and unlock a different ending.

2) Locations / Events

- **At least 5 different locations or major events** the player can visit/experience

    - A “major event” is a big moment in the story that changes what happens next. It is not just moving to a new room. It is something that forces the player to make a decision, face a challenge, or causes the game state to change.

    - Examples of major events:

        - An alarm is triggered and a countdown starts, you must choose where to run.

        - An NPC offers a deal, help them and gain an item, refuse and lose access to a shortcut.

        - A locked door forces you to solve a puzzle or find a key item before you can continue.

        - A boss/enemy encounter happens and your result changes what areas you can access.

        - The player finds a critical clue (a code, map, or message) that unlocks a new path or ending.

3) NPC Interactions

- **At least 5 NPCs (Non-Player Characters)**

- Each NPC must have dialogue and at least **one meaningful interaction**, such as:

    - giving a clue

    - trading or giving an item

    - triggering a challenge

    - changing the story path
    - Etc.....

4) Inventory System

Your game must include an inventory system where:

- The player can collect items (**minimum 5 items** total in the game)
- The player can **view inventory** at any time
- Items must have **a purpose**, such as:
    - unlocking a location
    - solving a puzzle
    - healing the player
    - influencing an ending
    - Using it for combat
    - Etc.....

5) Challenges (Puzzles / Obstacles / Combat)

- Include **at least 2 challenges** total
- Challenges can be puzzles, obstacles, enemy encounters, etc....

6) User Experience

Your game must have:

- Clear instructions for the player
- Input handled safely so the game does not crash with unexpected input
- A clean menu or clear prompts

<u>Example: Clear “One-Choice” Prompt (Terminal Menu)</u>

What would you like to do?

1. View Facility Map
2. Check Inventory
3. Attempt Door Unlock (Requires Keycard OR Access Code)
4. Read System Logs
5. Save Game

6. Quit

Enter a number (1-6): _

If the player enters something invalid (example: 9, or "hello"):

- Display: "Invalid choice. Please enter a number from 1 to 6."
- Then re-display the menu and ask again.

Part 2: Required Cybersecurity Element (The “Cyber Pack”)

Every team must implement all three Cyber Pack requirements below.

Cyber Pack Requirement 1: Input Validation + Safe Error Handling

- Your game must safely handle invalid inputs
- Use `try/except` where needed to prevent crashes (example: user types letters when a number is expected)
- The player should be re-prompted after invalid input

Cyber Pack Requirement 2: Audit Logging (Security Log File)

- Your game must write an audit log to a file named: **audit_log.txt**
- The log must include a timestamp and a short message for security-relevant events, such as:
    - game start / game end
    - save attempts and load attempts (success/failure)
    - challenge attempts (success/failure)
    - major story decisions (optional but recommended)
- Audit Log Examples (You can choose your own style)
- Your audit log should write one line per important event. Each line should include:
    - timestamp
    - event type
    - result (SUCCESS/FAIL) when applicable

    - short details (optional but helpful)

- Example lines:
    - 2026-01-19 14:35:21 - GAME_START - Player started a new game
    - 2026-01-19 14:36:02 - INPUT_INVALID - User entered "abc" at main menu
    - 2026-01-19 14:36:25 - CHOICE_MADE - Scene=ServerRoom - Choice=Read System Logs
    - 2026-01-19 14:37:10 - CHALLENGE_ATTEMPT - Puzzle=TerminalLogin - FAIL - Attempts=1
    - 2026-01-19 14:37:42 - CHALLENGE_ATTEMPT - Puzzle=TerminalLogin - SUCCESS - Attempts=2
    - 2026-01-19 14:38:05 - SAVE_ATTEMPT - SUCCESS - File=savegame.json
    - 2026-01-19 14:38:31 - LOAD_ATTEMPT - FAIL - Reason=SAVE_TAMPERED
    - 2026-01-19 14:39:12 - ITEM_USED - Item=Keycard - Result=DoorUnlocked
    - 2026-01-19 14:40:00 - GAME_END - Ending=MaintenanceTunnelEscape

Cyber Pack Requirement 3: Save / Load With Tamper Check

- Your game must support saving and loading game progress
- You must include a tamper-detection check so if someone edits the save file, the game can detect it

Recommended approach:

- Save game state (example: JSON or structured text)
- Compute a hash (example: SHA-256) of the save contents and store it alongside the save
- When loading, recompute the hash and confirm it matches
- If it does not match, treat the save as “tampered” and reject it or force a new game

Part 3: Team Roles (Teams of 6, Two Students Per Role)

Each team will have 3 roles, with 2 students assigned to each role. Every student must write code and contribute through GitHub pull requests.

Role 1 (2 students): Core Engine + Integration

Primary responsibility: Make the game run as a complete program and integrate all modules.

Code ownership:

- `main.py`
- `engine/game_engine.py`
- `engine/state.py` (or `engine/utils.py`)

Expected coding contributions:

- Main game loop (example: `while` loop), navigation between scenes
- Menu/choice handling and routing between story paths
- Connecting story, inventory, challenges, and security modules together
- Integration work: merging PRs, resolving conflicts, ensuring the game runs end-to-end

Role split suggestion:

- Student A: main loop + routing
- Student B: state management + integration support

Role 2 (2 students): Story + Gameplay Systems

Primary responsibility: Implement story content and the inventory system as code.

Code ownership:

- `content/story.py`
- `content/npcs.py`
- `systems/inventory.py`

Expected coding contributions:

- Story scenes structured as dictionaries/lists (not just raw text)
- Branching flags and conditions that affect outcomes
- NPC interactions implemented as functions (dialogue + choices + outcomes)
- Inventory system (add/remove/use items, item effects)

Role split suggestion:

- Student A: story scenes + branching logic/flags
- Student B: NPC interactions + inventory module

Role 3 (2 students): Challenges + Cybersecurity

Primary responsibility: Build challenges AND implement the Cyber Pack.

Code ownership:

- `systems/challenges.py`
- `security/security.py`
- `security/save_load.py` (optional)

Expected coding contributions:

- At least 2 challenges coded as functions (puzzle/combat/obstacle)
- Cyber Pack code:
  - input validation helpers used by the engine
  - audit logging to `audit_log.txt`
  - save/load with tamper check (hash integrity)

Role split suggestion:

- Student A: challenges
- Student B: cyber pack
- Both students: PR reviews and testing

Part 4: Required GitHub Collaboration Rules

This project requires GitHub teamwork.

Repository Rules

- **No direct pushes to the main branch**
- All code changes must go through pull requests (PRs)

Pull Requests (Required)

Each student must:

- Create **at least 2 pull requests**

- Have each PR reviewed and approved by at least **1 teammate** before merging

Commits (Required)

Each student must:

- Have **at least 6 commits**
- Commits must reflect real progress (not spam commits)

Issues (Required)

Each team must:

- Create **at least 8 GitHub Issues** (tasks/bugs/features)
- Assign issues to team members

Code Reviews (Required)

Each student must:

- Complete **at least 2 PR reviews** (approve or request changes with comments)

Notes

- "Real work" is required. Very small or fake commits/PRs will not count.
- You should spread contributions across the project, not upload everything on the last day.

Part 5: Submission Requirements

Team Submission (One Per Team)

Submit the following:

1. GitHub repository link
2. `docs/flowchart.png` (or PDF) showing story branching and endings which should be attached in your GitHub
3. A tagged release named: **v1.0** (or a clearly labeled final commit)
4. A `README.md` that includes:
    a. project name and team member names
    b. how to run the game
    c. list of core game features included

d. Story Paths and Endings

    i. List the 3 story paths (short description of each)

    ii. List the 3 endings (short description of each)

e. Locations / Events

    i. List the 5 locations or major events the player can visit/experience

f. List the 5 NPCs the player can encounter

    i. For each NPC, include:

        1. where they appear

        2. what they do (clue, item, challenge trigger, choice outcome, etc..

g. Inventory Items

    i. List the 5 items the player can collect

    ii. For each item, include what it is used for

h. List the 2 challenges and explain:

    i. where they occur

    ii. what the player must do

    iii. what happens on success vs failure

i. explanation of the Cyber Pack features and how they work

Individual Submission (Each Student)

- Each student must submit a short survey (5 sentences or one short paragraph is enough). Copy and answer the questions below.
    - What did you personally code?
        - List the specific files/modules you worked on and 2–3 functions or features you added.
        - Example: “I worked on systems/inventory.py and added add_item(), remove_item(), and use_item(). I also updated one NPC interaction in content/npcs.py.”
    - What was one teamwork challenge your group faced, and how did you solve it?
        - Example: “We had merge conflicts when two people edited the same file. We solved it by assigning file owners and using pull requests with code reviews.”
- Teammate contribution score (private, instructor only)
    - Give each teammate a score from 1–5 based on contribution:
        - 5 = excellent contribution

        - 4 = strong contribution
        - 3 = average contribution
        - 2 = limited contribution
        - 1 = little or no contribution
    - For each teammate, include one short reason (example: “completed assigned issues,” “submitted PRs,” “attended meetings,” “did not respond,” etc.).
    - Important: This peer scoring is private and will only be used to understand teamwork and individual effort.
    - Note: Individual private scores may affect your individual project grade. If multiple teammates give you a score of **1** and provide clear reasoning (example: no commits/PRs, missed meetings, did not complete assigned issues, lack of communication), your individual score WILL be adjusted to reflect your level of contribution.

Notes

- Every student must contribute code. Writing story text alone is not enough, story must be implemented using data structures and functions.
- Projects that crash on invalid input will lose points.
- Your grade includes both the team’s final product and your individual GitHub contributions.

Grading Breakdown (100 Points Total)

1) Functionality and Core Game Requirements (35 points)

- Story paths and endings work correctly (3 paths, 3 endings
- Locations/events implemented (5 total)
- NPC interactions implemented (5 total, meaningful interactions)
- Inventory system implemented (collect, view, use items; 5 items)
- Challenges implemented (2 total, success/fail outcomes)

2) Cyber Pack (20 points)

- Input validation + safe error handling (no crashing on invalid input)

- Audit logging to audit_log.txt (timestamped, security-relevant events)
- Save/load with tamper check (detects altered save files)

3) Code Quality and Organization (15 points)

- Code is readable (naming, formatting, comments)
- Modular design (organized files, functions, minimal repetition)
- Stability (runs smoothly, handles edge cases reasonably)

4) GitHub Collaboration Evidence (15 points)

- Individual contribution minimums met (2 PRs, 6 commits)
- PR reviews completed (at least 2 reviews per student)
- Issues used appropriately (at least 8 issues per team, assigned)

5) Documentation and Submission Quality (15 points)

- README includes the required checklist (locations, NPCs, items, challenges, paths/endings)
- Flowchart included in docs folder and matches the game